Universidade do Vale do Itajaí
Escola Politécnica
Curso de Ciência da Computação
**Algoritmos e Programação II (22817)**

---

*Definição do trabalho da M3 – BOMBERMAN: DUELO EM TEMPO REAL*

---

***Data da entrega:*** ___/___/_____
***Data da defesa:*** ___/___/_____
***Modalidade:*** em TRIO.

## *Visão Geral:*

O trabalho da M3 consiste em desenvolver, para o console, uma versão do clássico Bomberman para DOIS jogadores no mesmo computador, em tempo real: os dois jogadores jogam ao mesmo tempo, no mesmo teclado, em uma arena com paredes e blocos, e vencem colocando bombas de modo que a explosão atinja o adversário, sem serem atingidos.

Nos trabalhos anteriores, o jogo avançava apenas quando o jogador apertava uma tecla. Aqui o jogo não espera ninguém: bombas têm um pavio que queima enquanto os jogadores se movem, explosões duram um tempo e desaparecem, e a rodada tem um limite de tempo. Por isso, o trabalho junta o que a turma aprendeu nos últimos conteúdos com um assunto novo: o controle do tempo em C++.

**O que o trabalho exercita:** (1) **alocação dinâmica**: os mapas podem ter qualquer tamanho, e a quantidade de bombas e de chamas na arena não é conhecida de antemão; (2) **ponteiros**: o estado do jogo é manipulado por ponteiros e vetores dinâmicos; (3) **structs**: a equipe projeta, e justifica, as estruturas que representam a bomba, o jogador, o mapa e o jogo; (4) **arquivos**: mapas lidos de arquivos e histórico de partidas gravado; (5) **templates**: uma mesma função cuida do crescimento de vetores de tipos diferentes, e uma mesma função lê mapas escritos em dois formatos; (6) **tempo**: pavio, ritmo do jogo, intervalo entre passos e cronômetro da rodada.

## *O que muda em relação aos trabalhos anteriores:*

| Aspecto | Nos trabalhos anteriores | No Bomberman |
|---|---|---|
| Passagem do tempo | O jogo só avança quando uma tecla é pressionada | O jogo avança sozinho, em ciclos de tempo controlados (tempo real) |
| Jogadores | Um | Dois, simultâneos, no mesmo teclado |
| Tamanho do cenário | Matriz fixa, com limite máximo no código | Qualquer tamanho, lido do arquivo e alocado em tempo de execução |
| Quantidade de elementos | Fixa ou limitada por constante | Bombas e chamas: vetores dinâmicos que crescem e diminuem |
| Memória | Variáveis e vetores estáticos; ponteiros proibidos | Ponteiros, new e delete obrigatórios; nenhum vazamento |
| Arquivos | Mapas (.csv), solução, jogo salvo e ranking | Mapas (.txt) e histórico de partidas (.csv) |

## ***Regras do jogo:***

### A arena

A arena é uma grade de L linhas por C colunas, lida de um arquivo (veja a seção “Mapas em arquivo”). Cada célula contém um dos símbolos abaixo:

| Símbolo | Significado |
|---|---|
| `#` | Parede indestrutível. Nada a atravessa: nem jogadores, nem bombas, nem explosões. |
| `+` | Bloco destrutível. Impede a passagem, mas é destruído (vira célula vazia) quando atingido por uma explosão, que para nele. |
| `.` | Célula vazia. |
| `1` e `2` | Posição inicial dos jogadores 1 e 2. Cada símbolo aparece exatamente uma vez no mapa e, em jogo, a célula é tratada como vazia. |

### Movimento

- Cada jogador se move uma célula por vez, nas quatro direções, sem diagonais.
- Mesmo mantendo a tecla pressionada, cada jogador só pode dar um passo a cada **150 ms**. O intervalo vale para cada jogador separadamente: um jogador parado não atrasa o outro.
- Um jogador não pode entrar em uma parede, em um bloco, em uma célula com bomba nem na célula do outro jogador. Quem acabou de colocar uma bomba pode sair da célula dela normalmente.

### Bombas

- Uma tecla faz o jogador colocar uma bomba na célula em que está. Cada jogador pode ter, ao mesmo tempo, no máximo 3 bombas ativas, e não pode colocar uma bomba em uma célula que já tem outra.
- A bomba explode **3 segundos** depois de colocada. O pavio é medido em tempo, não em ciclos do jogo: em um computador mais lento ou mais rápido, a bomba deve explodir no mesmo instante (tolerância de 100 ms).
- A explosão ocupa a célula da bomba e se estende em cruz (para cima, para baixo, para a esquerda e para a direita) por até **2 células** em cada direção. Em cada direção, a explosão para ao encontrar uma parede (que não é atingida) ou um bloco (que é destruído e interrompe a explosão naquela direção).
- Cada célula atingida fica em chamas por **500 ms**. Um jogador que esteja em uma célula em chamas, ou que entre nela durante esse tempo, morre.
- **Explosão em cadeia:** se a explosão de uma bomba atinge outra bomba, essa outra explode imediatamente, no mesmo instante, e a sua explosão também se propaga, e assim por diante.

### Rodadas, partida e vitória

- Uma partida é composta de até 3 rodadas, todas no mesmo mapa. A cada rodada, o mapa volta ao seu estado inicial, as bombas e chamas são descartadas e os jogadores voltam às suas posições iniciais.
- A rodada termina quando resta no máximo um jogador vivo. Quem sobreviver vence a rodada. Se os dois morrerem no mesmo instante, ou se o tempo da rodada (2 minutos) acabar, a rodada é empatada e ninguém pontua.

- A partida termina quando um jogador vence 2 rodadas ou quando as 3 rodadas forem disputadas. Vence a partida quem tiver mais rodadas vencidas, e a partida pode terminar empatada.
- Ao final da partida, o resultado é gravado no histórico (veja a seção “Histórico de partidas”) e o jogo volta ao menu.

**Tela durante a rodada**

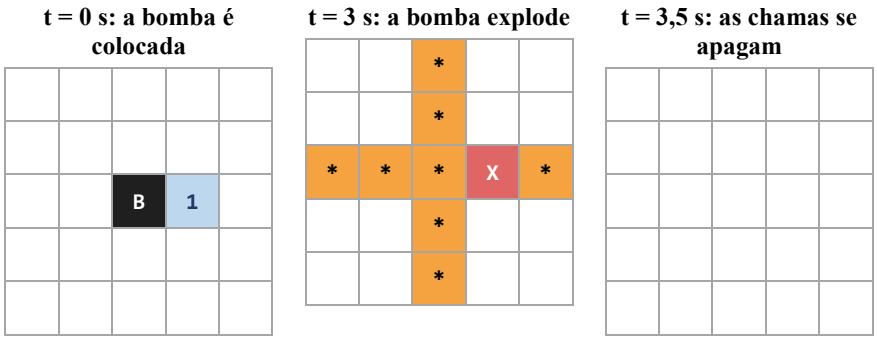
Além da arena, a tela deve mostrar, em todos os momentos: o nome e as rodadas vencidas de cada jogador, a quantidade de bombas que cada um ainda pode colocar, o número da rodada e o tempo restante da rodada no formato mm:ss. Jogadores, bombas, chamas, blocos e paredes devem ser distinguíveis na arena por símbolos diferentes, definidos pela equipe.

***Exemplos visuais da explosão:***

As figuras abaixo mostram como a explosão deve se comportar. Os símbolos e as cores são apenas ilustrativos: a equipe escolhe como desenhar cada elemento na tela do jogo. Em todos os exemplos, o alcance é de 2 células e o pavio é de 3 s.

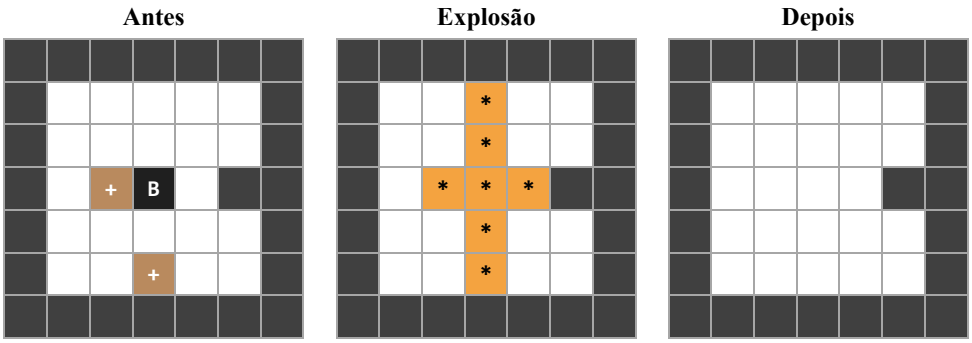



**Figura 1 – Uma bomba, do começo ao fim**




O jogador 1 colocou a bomba e ficou ao lado dela. Quando ela explodiu, a explosão ocupou a célula da bomba e, em cada direção, até 2 células. Como o jogador estava na linha da explosão, foi atingido. Se ele tivesse saído da cruz antes dos 3 s, sobreviveria. As chamas permanecem 0,5 s e depois a arena volta ao normal.

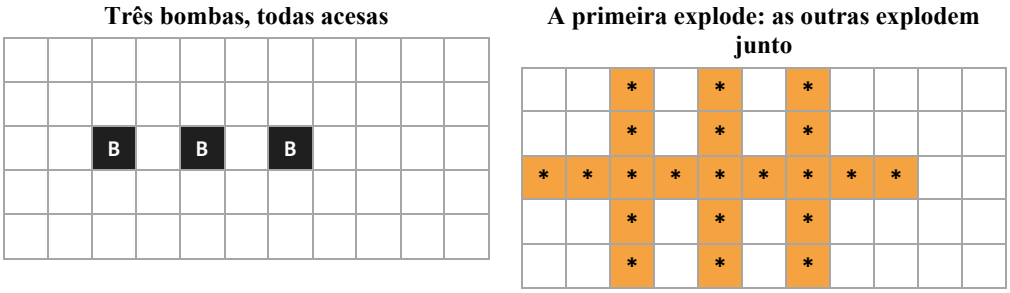
**Figura 2 – A explosão encontra paredes e blocos**




À direita da bomba, a primeira célula está livre e a segunda é uma parede: só a primeira entra em chamas, e a parede não é atingida. À esquerda, o bloco ocupa a primeira célula: ele é destruído, entra em chamas e interrompe a explosão naquela direção. Para baixo, a primeira célula está livre e a segunda tem um bloco, que também é destruído; a explosão alcançaria 2 células, e o bloco foi justamente a segunda. Para

cima, nada impede o alcance total. Depois que as chamas se apagam, os blocos destruídos viram células vazias.

**Figura 3 – Explosão em cadeia**




As bombas estão a 2 células de distância uma da outra. A explosão da primeira atinge a segunda, que explode no mesmo instante; a explosão da segunda atinge a terceira, e assim por diante. Todas as chamas aparecem no mesmo ciclo do jogo, e cada bomba explode uma única vez, mesmo que seja atingida por mais de uma explosão.

## ***Controles:***

| Ação | Jogador 1 | Jogador 2 |
|---|---|---|
| Mover | W, A, S, D | Setas do teclado |
| Colocar bomba | F | Enter |
| Pausar / continuar | P (qualquer jogador) | |
| Abandonar a partida | ESC (pede confirmação) | |

Os dois jogadores usam o teclado ao mesmo tempo. A leitura de teclas com getch() lê uma tecla de cada vez e se comporta mal quando dois jogadores pressionam teclas simultaneamente; a apresentação sobre controle de tempo mostra como ler o estado de várias teclas ao mesmo tempo (GetAsyncKeyState).

## ***O tempo no jogo:***

Todos os intervalos abaixo devem ser definidos como constantes no código, em milissegundos, e todos devem ser medidos com o relógio (biblioteca chrono), nunca contando voltas de laço. Os valores são fixos, mas o jogo deve funcionar com qualquer um deles alterado.

| Intervalo | Valor | Observação |
|---|---|---|
| Ciclo do jogo | 33 ms | Cerca de 30 ciclos por segundo (leitura de teclas, atualização e desenho). Se um ciclo demorar mais que isso, o próximo não pode esperar. |
| Intervalo entre passos | 150 ms | Por jogador. |
| Pavio da bomba | 3000 ms | Medido desde o instante em que a bomba foi colocada. |
| Duração das chamas | 500 ms | Medido desde o instante da explosão (inclusive nas explosões em cadeia). |
| Tempo da rodada | 120000 ms | Exibido em contagem regressiva. |

**Pausa:** enquanto o jogo estiver pausado, nada do jogo avança: nem os pavios, nem as chamas, nem o cronômetro da rodada, nem o intervalo entre passos. Uma bomba que tinha 1 s de pavio restante quando o jogo foi pausado deve ter exatamente 1 s restante quando o jogo for retomado, seja a pausa de 2 s ou de 2 minutos.

**Instantes:** a forma de representar e de guardar os instantes (por exemplo, em milissegundos de um relógio do jogo, ou como pontos no tempo de um relógio da biblioteca) é decisão da equipe e deve ser justificada, principalmente em relação à pausa.

**Tela sem piscar:** o desenho da arena a cada ciclo não pode fazer a tela piscar. O uso de system("cls") a cada ciclo não é aceito; a apresentação mostra a técnica para redesenhar sobre a própria tela.

***Projeto das estruturas de dados (structs):***

O estado de uma partida deve ser mantido em structs, e não em variáveis soltas. Diferentemente do trabalho anterior, o enunciado não fornece as structs: o projeto delas faz parte do trabalho. Cada equipe decide quais structs criar, quais campos cada uma terá, de que tipos, como elas se relacionam e quem guarda o quê, e deve justificar essas decisões no README e na apresentação. O projeto deve ser capaz de representar, no mínimo, o que a tabela abaixo descreve:

| Elemento | O que precisa ser representado |
|---|---|
| Mapa | as dimensões, o título e o conteúdo de cada célula da arena; |
| Jogador | nome, posição, se está vivo, rodadas vencidas, quantas bombas tem ativas e quando poderá dar o próximo passo; |
| Bomba | onde está, a quem pertence, o alcance da sua explosão e quando foi colocada; |
| Chama | onde está e quando se apaga; |
| Partida | tudo o que é preciso para jogar e também para pausar, retomar e reiniciar uma rodada: mapa, jogadores, bombas, chamas, a passagem do tempo, a pausa e o número da rodada; |
| Registro do histórico | os dados de uma partida finalizada. |

A tabela diz o que representar, e não como. A equipe pode juntar elementos em uma struct, separá-los em mais structs ou criar structs que não estão na tabela, mas cada struct deve existir por um motivo que a equipe saiba explicar.

- **README:** o arquivo deve ter uma seção “Decisões de projeto das structs” que diga, para cada struct: por que ela existe; por que cada campo existe e por que tem aquele tipo; o que foi calculado, ou guardado em outro lugar, em vez de ficar na struct (e por quê); e que alternativa foi considerada e descartada.
- Na apresentação, o professor poderá perguntar, por exemplo: por que este dado está nesta struct e não em outra? Por que a bomba guarda este campo e não aquele? O que precisaria mudar no código para acrescentar um novo elemento ao jogo (como um item)?
- Não use variáveis globais. Constantes (como os intervalos de tempo) são permitidas.
- Pelo menos uma sub-rotina deve receber a struct que reúne o estado da partida por ponteiro e acessar seus campos com o operador `->`. As demais podem usar ponteiro ou referência, e a escolha deve ser justificada.

***Alocação dinâmica:***

O trabalho não pode conter nenhum limite fixo de tamanho de mapa, de quantidade de bombas ou de quantidade de chamas escrito no código (como uma constante MAX). Os pontos obrigatórios de alocação dinâmica são:

- **O mapa:** a matriz de células é alocada com as dimensões lidas do arquivo (linha por linha ou em bloco único) e liberada ao fim da partida.
- **As bombas:** vetor dinâmico, que começa com capacidade 2 e **dobra** de capacidade quando fica cheio (aloca um vetor maior, copia o conteúdo, libera o antigo). Quando uma bomba explode, ela é removida do vetor.
- **As chamas:** vetor dinâmico com o mesmo comportamento, com chamas removidas quando se apagam.
- **O histórico:** o arquivo é lido para um vetor alocado com exatamente a quantidade de registros do arquivo (veja a seção do histórico).

O crescimento e a remoção dos vetores devem ser feitos por funções template, escritas uma única vez e usadas com pelo menos dois tipos diferentes de struct (as bombas e as chamas).

**Nenhum vazamento é aceito:** todo new deve ter o seu delete em todos os caminhos do programa, inclusive ao abandonar a partida com ESC, ao trocar de rodada e ao voltar ao menu. Cada vetor deve ter um responsável claro por criá-lo e por liberá-lo, descrito no README.

***Mapas em arquivo:***

Os mapas ficam na pasta de execução, com os nomes `mapa1.txt`, `mapa2.txt`, `mapa3.txt` e assim por diante. O jogo descobre os mapas existentes tentando abrir um arquivo após o outro, até que um deles não exista; não pode haver um número de mapas escrito no código. O formato dos arquivos é padronizado, e todas as equipes devem ler os arquivos do mesmo modo. Um mapa pode estar escrito em um de **dois formatos**, indicado na primeira linha do arquivo, e o jogo deve aceitar os dois:

- linha 1: a quantidade de linhas `L`, a quantidade de colunas `C` e a letra do formato, separados por espaços: `C` (mapa em caracteres) ou `N` (mapa em números);
- linha 2: o título do mapa (texto livre, sem acentos), exibido na escolha de mapa;
- linhas 3 até `L+2`: as `L` linhas da arena, cada uma com exatamente `C` valores, escritos conforme o formato.

| Elemento | Formato C (caracteres) | Formato N (números) |
|---|---|---|
| Parede indestrutível | `#` | `1` |
| Bloco destrutível | `+` | `2` |
| Célula vazia | `.` | `0` |
| Posição inicial do jogador 1 | `1` | `3` |
| Posição inicial do jogador 2 | `2` | `4` |

No formato C, os símbolos de uma linha vêm juntos, sem separador. No formato N, os números de uma linha são separados por um espaço. A arena é a mesma nos dois formatos: apenas a forma de escrevê-la muda. Exemplos, fornecidos junto com este enunciado (mapa_exemplo_C.txt e mapa_exemplo_N.txt):

```
11 13 C
Arena Classica
#############
#1.++.++++++#
#.#+#+#+#+#.#
#++..++.+.++#
#+#+#.#+#+#.#
#+++++.+++++#
#.#.#+#+#+#.#
#.+.++.+++..#
#+#.#+#.#+#.#
#++..+.+...2#
#############
```

```
11 13 N
Arena Classica Numerica
1 1 1 1 1 1 1 1 1 1 1 1 1
1 3 0 2 2 0 2 2 2 2 2 2 1
1 0 1 2 1 2 1 2 1 2 1 0 1
1 2 2 0 0 2 2 0 2 0 2 2 1
1 2 1 2 1 0 1 2 1 2 1 0 1
1 2 2 2 2 2 0 2 2 2 2 2 1
1 0 1 0 1 2 1 2 1 2 1 0 1
1 0 2 0 2 2 0 2 2 2 0 0 1
1 2 1 0 1 2 1 0 1 2 1 0 1
1 2 2 0 0 2 0 2 0 0 0 4 1
1 1 1 1 1 1 1 1 1 1 1 1 1
```

O jogo deve funcionar com mapas de qualquer dimensão, desde um mapa pequeno (5 x 5) até um grande (por exemplo, 25 x 60). Quem escreve o mapa é responsável por escolher dimensões que caibam na janela do console; o jogo não precisa tratar isso.

**Dois formatos, uma só função de leitura (template)**

A leitura das células dos dois formatos deve ser feita por uma única função template, parametrizada pelo tipo da célula (char para o formato C e int para o formato N), sem duplicar em duas funções a lógica de leitura e de validação. O restante do jogo não deve saber em que formato o mapa foi escrito: depois de lido, mapas dos dois formatos devem ser tratados exatamente do mesmo modo. A equipe decide como as células são representadas na memória e em que momento os valores do formato N são traduzidos, e deve justificar essa decisão.

- Dica: de um formato para o outro, muda apenas o tipo do valor lido de cada posição da linha. Pense em como ler a linha inteira e extrair dela os valores com o operador >> sobre uma variável do tipo T.

**Validação na leitura**

Ao carregar um mapa, o jogo deve recusá-lo, informando o motivo, nos casos abaixo. Um mapa recusado aparece na escolha de mapas como indisponível, com o motivo, e nunca derruba o programa:

- primeira linha ausente, com L ou C não numéricos ou menores que 5, ou com letra de formato diferente de C e N;

- título ausente, menos linhas do que L, ou linha com quantidade de valores diferente de C;
- valor fora do conjunto do formato: símbolo diferente de #, +, ., 1 e 2 (formato C) ou número diferente de 0, 1, 2, 3 e 4 (formato N);
- não haver exatamente um jogador 1 e exatamente um jogador 2.

**Dicas sobre arquivos**

- Para ler os valores da primeira linha e passar para a linha seguinte, use `arq >> L >> C >> formato;` seguido de `getline(arq, resto);` (que consome o final da linha), e só depois `getline(arq, titulo);`.
- Para saber se um arquivo existe, tente abri-lo e teste `is_open()`. Se a validação falhar no meio da leitura, libere a memória que já foi alocada para o mapa antes de devolver o erro.
- Uma função que lê arquivo não deve imprimir na tela. Para informar o motivo de uma falha, devolva um bool e preencha um parâmetro string, por referência, com a mensagem.

## ***Histórico de partidas:***

Ao final de cada partida, o jogo acrescenta uma linha ao final de `historico.csv`, no formato `nome1,nome2,mapa,vitorias1,vitorias2,segundos`, onde `segundos` é a duração total da partida (sem contar o tempo de pausa). Exemplo:

```
Ana,Bruno,1,2,0,171
Bruno,Ana,3,1,2,298
```

A opção "Histórico" do menu conta as linhas do arquivo, aloca um vetor de structs com exatamente essa quantidade, lê o arquivo para o vetor e exibe as 10 partidas mais recentes e o total de partidas já jogadas. O arquivo pode não existir (nenhuma partida jogada ainda): nesse caso, o jogo informa isso e continua normalmente. Os nomes dos jogadores têm até 10 caracteres, sem vírgulas e sem espaços.

## ***Menu:***

O jogo deve possuir um menu com as opções abaixo. O programa só é encerrado pela opção "Fim"; qualquer outra opção volta ao menu ao terminar.

| Opção | Exigências |
|---|---|
| Jogar | Lista os mapas descobertos pelo título (os inválidos aparecem como indisponíveis, com o motivo). O jogador escolhe um mapa específico ou um mapa aleatório entre os válidos. Em seguida, os dois jogadores informam seus nomes e a partida começa. |
| Histórico | Exibe as 10 partidas mais recentes e o total de partidas, a partir de historico.csv. |
| Como jogar | Explica as regras do jogo e mostra as teclas de cada jogador. |
| Sobre | Exibe a equipe (nome de cada integrante), o mês/ano e o nome do professor/disciplina. |
| Fim | Encerra o programa. É a única opção que o encerra. |

***Regras para o desenvolvimento:***

- O jogo é desenvolvido em C++, para console, sem bibliotecas gráficas nem motores de jogos. O programa deve compilar e executar no Windows, como nos computadores do laboratório. São permitidas as bibliotecas padrão e, para o teclado e para o console, conio.h e windows.h.
- Não é permitido o uso de variáveis globais, de classes, nem da classe vector (nem de qualquer contêiner da biblioteca padrão que faça a alocação por você): a alocação e a liberação de memória devem ser feitas pela equipe.
- Recursos esperados: funções, passagem de parâmetros por valor, por referência e por ponteiro, vetores e matrizes dinâmicos, structs, arquivos (fstream), funções template e a biblioteca `<chrono>` para o tempo.
- O tempo deve ser controlado com a biblioteca chrono (preferencialmente steady_clock), comparando o instante atual com o instante registrado. Contar voltas de laço ou movimentos não é aceito. Usar Sleep ou sleep_for para esperar é permitido apenas para completar o tempo do ciclo do jogo; esperar com Sleep em outros pontos (por exemplo, até a bomba explodir) trava o jogo e não é aceito.
- A função main deve ser curta: essencialmente o menu e a chamada das sub-rotinas. O laço do jogo também deve ficar em uma sub-rotina.
- Cada sub-rotina deve cuidar de **uma só** das responsabilidades: aplicar as regras (altera o jogo, não imprime e não acessa arquivo), desenhar a tela (imprime, não altera) ou acessar arquivos (lê e grava, não imprime e não aplica regras).
- Arquivos ausentes, vazios ou com conteúdo inválido nunca podem travar nem encerrar o programa de forma inesperada.
- Produzam todo o código pensando na possibilidade de que novas funcionalidades poderão ser solicitadas no futuro.

***Mapas do trabalho:***

- O trabalho deve acompanhar no mínimo 5 mapas: os dois mapas de exemplo fornecidos e pelo menos 3 mapas autorais, independentemente do número de integrantes. Entre os autorais deve haver pelo menos um em cada formato (C e N).
- Entre os mapas autorais deve haver pelo menos um com dimensões diferentes de 11 x 13, um pequeno (até 7 x 9) e um grande (pelo menos 17 x 31), para demonstrar que o jogo não depende de um tamanho fixo.
- Os mapas autorais devem ser submetidos ao professor antes da implementação, que garantirá que nenhuma equipe implemente um mapa semelhante ao de outra.
- Os dois jogadores devem começar com as mesmas chances. Garanta que cada um tenha pelo menos 3 células livres próximas à sua posição inicial, para poder colocar uma bomba e escapar.

***Sobre o uso de sub-rotinas:***

Continua sendo o principal critério técnico do trabalho. Espera-se, no mínimo, sub-rotinas para:

- descobrir quantos mapas existem, carregar um mapa devolvendo o motivo quando inválido, e liberar um mapa;
- iniciar uma rodada (volta ao estado inicial) e encerrar uma rodada (libera o que foi alocado);
- ler o teclado dos dois jogadores e transformar o que foi lido em ações;

- mover um jogador, respeitando o intervalo entre passos e as colisões;
- colocar uma bomba, verificar quais bombas devem explodir e explodir uma bomba (incluindo a explosão em cadeia);
- atualizar o relógio do jogo (levando em conta a pausa) e verificar o fim da rodada;
- desenhar a arena e a informação do jogo;
- gravar o resultado no histórico e carregar o histórico;
- ler as células de um mapa, nos dois formatos (função template), e traduzi-las, se necessário;
- crescer e remover elementos de um vetor dinâmico (funções template).

***Desafios de projeto:***

Há decisões sem resposta única, que fazem parte da avaliação e devem ser tomadas e justificadas na apresentação:

- Pausa: como o tempo em que o jogo ficou pausado deixa de contar para as bombas, para as chamas e para o cronômetro? Quais são as vantagens de um relógio do jogo, separado do relógio real?
- Ponteiros e realocação: ao dobrar a capacidade do vetor de bombas, o vetor antigo é liberado. O que acontece com um ponteiro ou uma referência para uma bomba que tenha sido guardado antes do crescimento? Como o código evita esse problema?
- Remoção: como remover uma bomba do meio do vetor? A ordem das bombas no vetor importa? Qual o custo de cada alternativa?
- Explosão em cadeia: como garantir que toda a cadeia seja resolvida no mesmo ciclo, sem explodir a mesma bomba duas vezes e sem remover elementos do vetor enquanto ele está sendo percorrido?
- Crescimento: por que dobrar a capacidade, em vez de aumentá-la de uma em uma posição? Quantas cópias cada estratégia faz até chegar a 1000 elementos?
- Reinício de rodada: recarregar o mapa do arquivo ou guardar uma cópia do mapa original na memória? Qual é o custo de cada uma e o que pode dar errado?
- O que acontece com o jogo se um ciclo demorar 500 ms (por exemplo, porque o computador travou por um instante)? Os jogadores devem andar várias casas de uma vez?

***Desafios extras (opcionais):***

Quem concluir todo o conteúdo obrigatório pode implementar um ou mais dos desafios abaixo. Cada desafio concluído e apresentado vale até 5% adicionais na nota do trabalho, limitada ao máximo de 100%:

- **Itens:** ao ser destruído, um bloco pode revelar um item (mais alcance de explosão, mais uma bomba disponível ou passos mais rápidos). Os itens também ficam em um vetor dinâmico.
- **Morte súbita:** nos últimos 30 segundos da rodada, paredes começam a fechar a arena a partir das bordas, a intervalos regulares.
- **Cores:** arena e jogadores coloridos, usando as funções de cor do console do Windows.
- **Adversário controlado pelo computador:** um modo em que o jogador 2 é controlado pelo programa, decidindo seus passos e bombas a cada ciclo.
- **Ranking por nome:** uma opção do menu que soma, a partir do histórico, as vitórias de cada nome de jogador e exibe a lista ordenada.

## ***ENTREGA:***

- Postar um arquivo texto com o link para o trabalho no GitHub. O histórico de commits deve mostrar a participação de todos os integrantes ao longo do desenvolvimento.

O repositório deve conter, além do código:

- **README.md** com: (a) como compilar e executar; (b) a tabela dos intervalos de tempo usados no jogo, com os valores das constantes; (c) a seção "Decisões de projeto das structs", com o projeto e a justificativa de cada struct; (d) a lista de todos os pontos de alocação dinâmica do programa, indicando quem aloca e quem libera cada um; (e) as decisões sobre a leitura dos dois formatos de mapa; (f) o que foi feito dos desafios extras, se houver.
- Os arquivos `mapaN.txt` de todos os mapas (mínimo 4).

## ***APRESENTAÇÃO:***

- A apresentação e a nota da apresentação serão individuais; além disso, todas as notas relativas ao código dependem do desempenho na apresentação. Sem apresentação, o trabalho terá nota ZERO.
- No dia da defesa, o professor executará o jogo da equipe com: (a) um mapa válido de autoria dele, de dimensão não usada pela equipe, em cada um dos dois formatos; (b) um arquivo de mapa inválido; (c) sem o arquivo historico.csv. O jogo não pode travar e deve tratar todos os casos.
- O professor medirá o pavio da bomba com um cronômetro e testará a pausa: o jogo será pausado com uma bomba acesa e retomado depois de alguns segundos.
- O professor sorteará uma função do código e pedirá ao aluno que a explique, inclusive quem aloca e quem libera a memória que ela utiliza.
- O aluno deverá explicar e justificar as decisões dos desafios de projeto.
- O aluno deverá justificar o projeto das suas structs (campos, tipos e relações entre elas).
- O professor poderá pedir pequenas modificações no código durante a apresentação (por exemplo, alterar um valor ou uma regra do jogo), para que o aluno demonstre domínio do código que apresenta.

## ***Critérios de Avaliação:***

1. **Organização e clareza do código = 10% da nota** (inclui o README e a separação entre regras, tela e arquivos).
2. **Funcionamento correto conforme a especificação = 20% da nota** (inclui o controle do tempo, a robustez diante de arquivos ausentes ou inválidos e o funcionamento com os mapas do professor).
3. **Recursos da linguagem utilizados, com destaque para alocação dinâmica, ponteiros, structs, templates e arquivos = 20% da nota** (inclui o projeto e a justificativa das structs).
4. **Apresentação do código = 50% da nota e é individual**.

*Obs1:* Cópia de soluções de colegas ou da internet acarretarão nota 0 para todos os envolvidos.

*Obs2:* Trabalhos entregues e/ou apresentados em atraso receberão um desconto inicial de 20% na nota final, além de um desconto progressivo de 10% por aula.